Monitor de vuelos Buenos Aires -> Río / São Paulo

Resumen ejecutivo: qué hace, cómo decide y cómo está armado · repositorio *monitor-vuelos-rio*

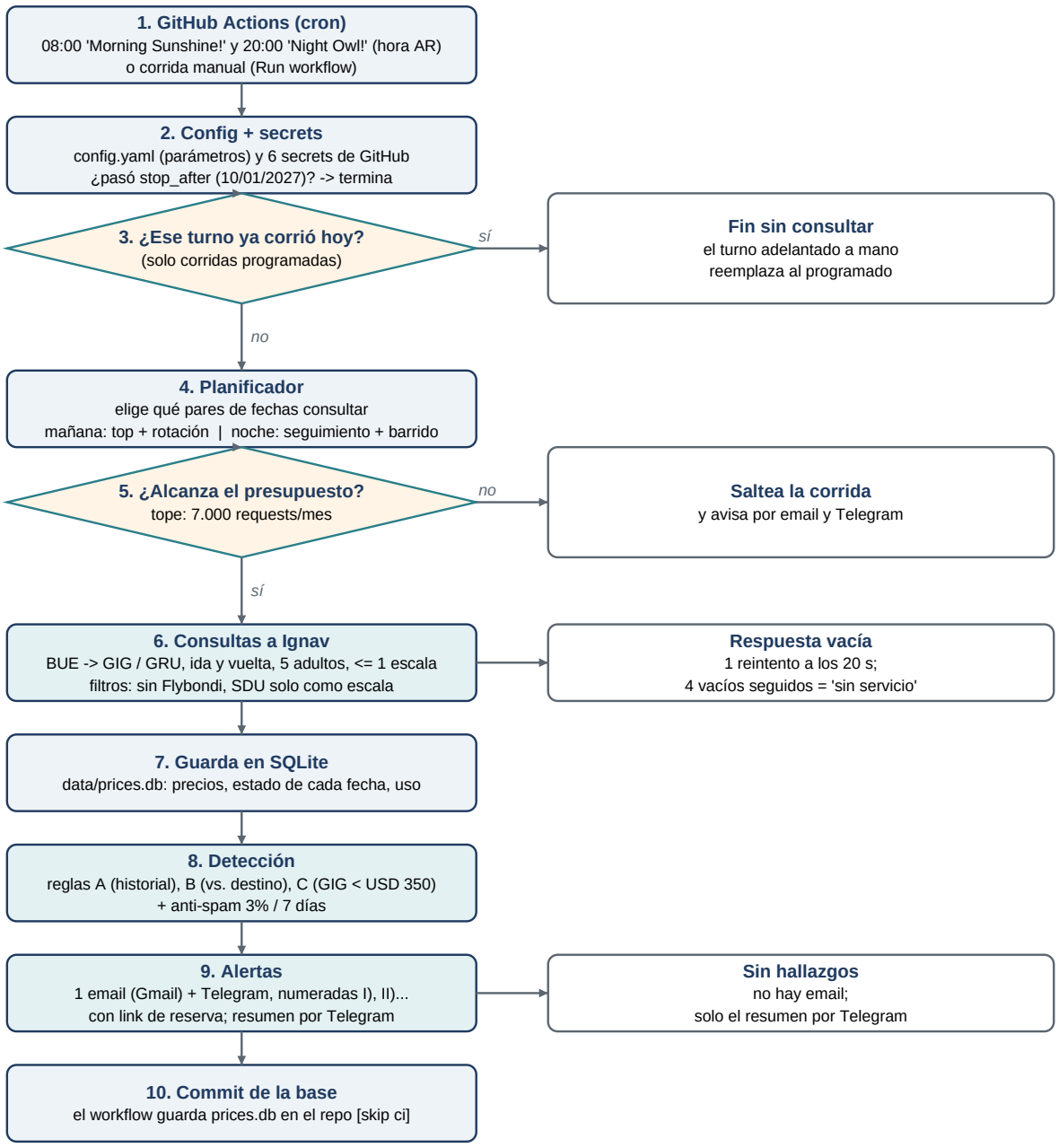
Qué es

Un bot que **dos veces por día** consulta precios de pasajes ida y vuelta de Buenos Aires a **Río de Janeiro (GIG)** y **São Paulo (GRU)** para 5 adultos, guarda el historial y **avisa por email y Telegram cuando aparece un precio bajo**. Es **100% determinístico**: reglas fijas y estadística simple, sin inteligencia artificial en tiempo de ejecución. Corre gratis en GitHub Actions; el único costo es la API de precios (Ignav).

Qué busca

Parámetro	Valor actual	Parámetro	Valor actual
Origen	BUE (Ezeiza y Aeroparque juntos)	Pasajeros	5 adultos, económica
Destinos	GIG (Galeão) y GRU (Guarulhos)	Escalas	máximo 1
Fechas	ida desde 10/01/2027, vuelta hasta 06/02/2027	Excluidos	Flybondi; Santos Dumont (SDU) solo como escala
Duración	9, 10 u 11 días (54 combinaciones por destino)	Moneda	USD, precio total con impuestos
Horarios	08:00 y 20:00 hora argentina	Fin	deja de consultar después del 10/01/2027

Flujo de una corrida



Cómo decide qué consultar (planificador)

Cada consulta a Ignav es **un par de fechas** (ida y vuelta) para un destino. Hay 54 pares por destino y cada consulta cuesta, así que el planificador reparte el presupuesto:

Turno	Río (GIG) - prioridad	São Paulo (GRU)
Mañana (08:00)	Todas sus fechas con vuelos	Las 12 más baratas + 10 de las menos actualizadas
Noche (20:00)	Todas otra vez + seguimiento de los hallazgos de la mañana	Las fechas que la mañana no miró + seguimiento de los hallazgos de la mañana
Siempre	Una fecha que vuelve vacía 4 veces seguidas se marca 'sin servicio' y se reintenta cada 2 días	Igual

Cómo decide si un precio es una oferta (detección)

Cada precio nuevo se compara de tres formas; con que se cumpla **una** regla hay alerta (una sola por itinerario). Las referencias son **medianas**, no promedios, para que un precio atípico no genere falsas alarmas.

Regla	Pregunta que responde	Condición
A - Historial	¿Está barato para esta misma fecha?	Con al menos 7 precios anteriores: precio < mediana x 0,85, o z robusto < -2,5 (z = (precio - mediana) / (1,4826 x MAD))
B - Destino	¿Es de lo más barato del destino ahora?	Está entre las **3 fechas más baratas** del destino y al menos 15% debajo de la mediana del destino (con 10 o más fechas con precio)
C - Precio fijo	¿Baja de un precio objetivo?	Río: precio por persona **menor a USD 350**
Anti-spam	¿Ya te lo avisé?	Una fecha ya alertada solo vuelve a avisar si bajó 3% más que la última alerta de los últimos 7 días
Empates	¿Varias fechas al mismo precio?	Se avisa la primera y se indica cuántas más tienen ese precio

Cómo avisa (alertas)

- **Email** (Gmail): uno por corrida, solo si hay hallazgos. Título 'Alerta Diaria de Vuelos by BotRabo - Morning Sunshine! / Night Owl! - N hallazgos'. Hallazgos numerados I), II)... ordenados del más barato al más caro, con fechas, escalas, aerolínea, precio por persona y total, link de reserva y estadísticas. Imagen al pie.
- **Noche**: el email empieza con el estado de cada hallazgo de la mañana ('El hallazgo II) de la mañana mantiene su precio / subió / bajó / ya no aparece') y después solo trae **fechas nuevas**.
- **Telegram**: la misma lista en formato corto y, en cada corrida, un resumen (consultas, top 5 y medianas).
- **Link de reserva**: se pide a Ignav solo para los itinerarios alertados (ahorra requests).

Datos, costo y seguridad

Tema	Cómo funciona
Persistencia	Base SQLite **data/prices.db**, que el propio workflow commitea al repo después de cada corrida. Guarda el mejor precio de cada consulta, el estado de cada fecha, cada request y cada alerta enviada.
Presupuesto	Antes de consultar estima los requests; si superaría el tope mensual (7.000) saltea la corrida y avisa. Cada request queda registrado aunque la corrida se corte.
Costo	Ignav: USD 2 cada 1.000 requests (los primeros 1.000 fueron gratis). Uso estimado ~5.500 por mes = ~USD 11. GitHub Actions: sin costo.
Seguridad	Las 6 claves (Ignav, Telegram, Gmail, destinatarios) son secrets de GitHub: nunca están en el código ni en config.yaml, y el log reemplaza cualquier clave por ***.
Calidad	90 tests automáticos con la API simulada (nunca llaman a Ignav ni mandan mensajes). Se corren en cada cambio (workflow tests.yml).
Operación manual	Desde GitHub -> Actions -> Run workflow: elegir turno (morning/night), dry_run (4 consultas, no alerta), full_scan, test_alert (prueba de canales), demo_alert (ejemplo).

Qué contiene cada carpeta y archivo

Ruta	Contenido
config.yaml	**Todos los parámetros**: fechas, destinos, pasajeros, escalas, exclusiones, cuántas fechas consultar por turno, umbrales de las reglas, tope mensual, canales y nombres de aerolíneas.
.github/workflows/monitor.yml	La automatización: horarios (cron), botón de corrida manual, ejecución del bot y commit de la base.
.github/workflows/tests.yml	Corre los tests en cada cambio del código.
monitor/cli.py, __main__.py	Punto de entrada (*python -m monitor*) y opciones: --slot, --dry-run, --full-scan, --test-alert, --demo-alert.
monitor/runner.py	**El orquestador**: ejecuta los pasos 3 a 9 del diagrama de flujo.
monitor/planner.py	Elige qué fechas consultar en cada turno (top, rotación, barrido, seguimiento).
monitor/sources/	Conexión con la fuente de precios. *base.py* define la interfaz común; *ignav.py* llama a la API de Ignav, filtra resultados y pide links de reserva. Otra fuente se agrega sin tocar el resto.
monitor/detection.py	Reglas A, B, C, anti-spam y estadísticas (mediana, MAD, z robusto).
monitor/storage.py	Base SQLite: tablas, migraciones y consultas (historial, ventana, uso mensual).
monitor/alerts/	*format.py*: textos de email, Telegram y resúmenes. *channels.py*: envío por Gmail y Telegram (si uno falla, sigue el otro).
monitor/assets/	Imagen que va al pie de cada email.
monitor/config.py, dates.py, logsetup.py, demo.py	Lectura y validación de config y secrets; generación de los pares de fechas; logs con claves ocultas; alertas simuladas.
data/prices.db	La base con todo el historial (la actualiza el workflow).
tests/	90 tests y respuestas de ejemplo de Ignav (*fixtures/*).
README.md, requirements*.txt, pyproject.toml	Manual completo y dependencias (requests, PyYAML; pytest para tests).

Argumentos para defender el diseño

- **Determinístico y auditable:** mismas entradas, misma decisión. Cada alerta guarda qué regla la disparó y cada request queda registrado, así que todo se puede reconstruir.
- **Eficiente:** no consulta todo siempre. Prioriza las fechas baratas (donde están las ofertas) y asegura que cada fecha se revise al menos una vez por día. Con un tope de gasto que no se puede superar.
- **Robusto a datos raros:** usa medianas y z robusto (MAD), que no se mueven con un precio atípico.
- **Sin spam:** no repite el mismo aviso salvo que el precio baje 3% más o pase una semana; agrupa empates.
- **Tolerante a fallas:** reintentos ante errores de la API, un canal caído no frena al otro, y la base registra el uso aunque la corrida se corte.
- **Configurable sin programar:** casi todo se cambia en config.yaml; los tests validan la configuración.

Limitaciones conocidas (para no ser sorprendido)

- **Depende de Ignav:** desde el 24/10 Ignav devuelve vacío para salidas a Río anteriores al 21/01, aunque São Paulo responde normal. El bot lo detecta y reintenta, pero no puede inventar precios.
- **Solo ida y vuelta al mismo destino:** combinaciones como ida a São Paulo y vuelta desde Río no se consultan (el adaptador de Ignav no lo soporta todavía).
- **Una foto por turno:** guarda el precio más barato de cada fecha en ese momento; una tarifa que dura menos de 12 horas puede pasar sin verse, y un link puede agotarse antes de que se abra (5 asientos).
- **Los precios suben con la fecha:** en temporada alta, las reglas relativas (A y B) avisan 'lo más barato del momento'; la regla C (USD 350 en Río) da un umbral absoluto.